

자연어처리와 컴퓨터비전 심층학습

10장

📅 날짜	2026년 5월 5일
🕒 발표팀	3팀
⚙️ 상태	시작 전
🏷️ 태그	9주차

ViT 개관

이미지 인식을 위한 딥러닝 모델 → 자연어 처리 방식처럼 분류해보려는 시도

셀프 어텐션을 사용하여 전체 이미지를 한 번에 처리하는 방식으로 구현함 트랜스포머 구조를 컴퓨터 비전 분야에 적용한 첫 번째 연구임.

문제: 이미지 패치가 순차적으로 입력돼서 이미지 특성을 온전히 반영한다고 보기 어려움

→ 스윈 트랜스포머 도입

로컬 윈도우를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지의 특징을 학습시킴.

→ CvT ehdlq

저수준 특징이 고수준 특징을 계층적으로 반영할 수 있음. 쿼리, 키, 값 중 키와 값을 기존 특징 벡터보다 축소해 계산 복잡도 감소

ViT

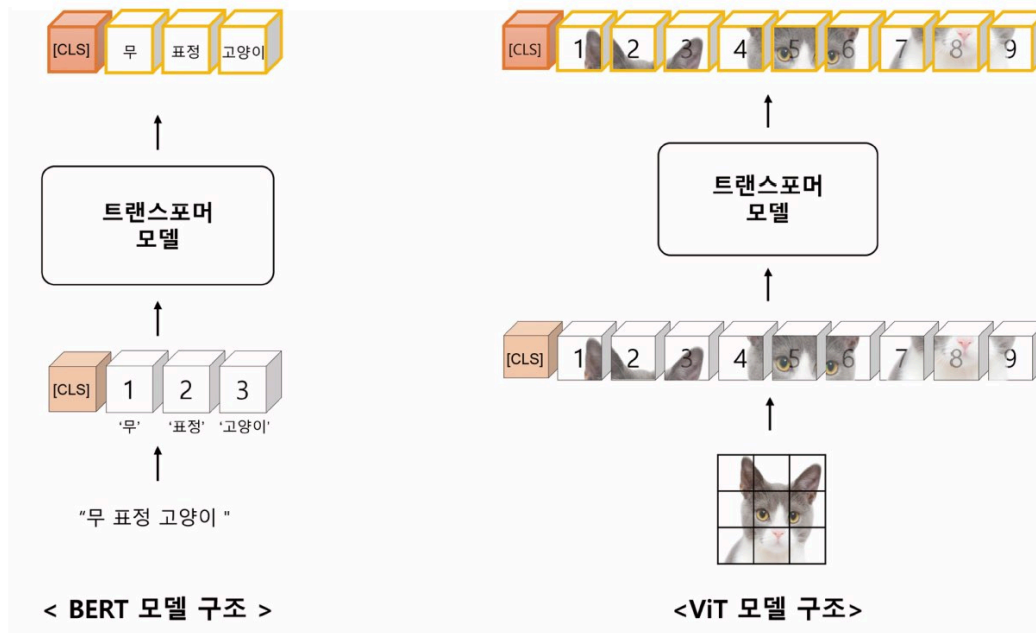


그림 10.2 BERT의 ViT 모델 구조 차이

트랜스포머 모델은 동일하지만 입력데이터를 만드는 과정이 서로 다르다고 할 수 있음. vit에서 사용되는 입력 이미지 패치는 왼→오 위→아래로 표현된 배열을 가정함.

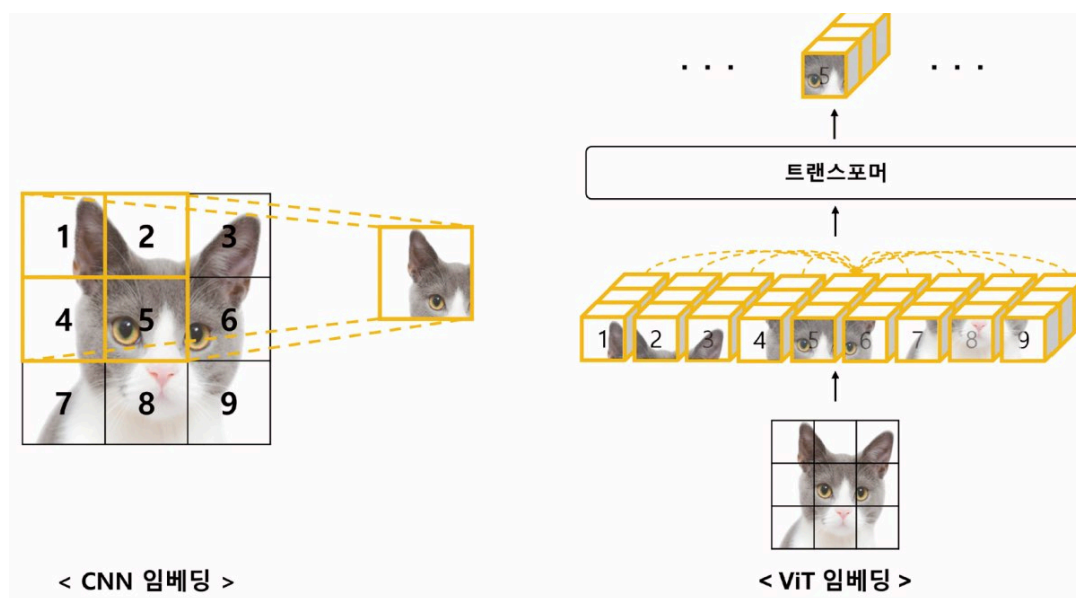


그림 10.3 합성곱 신경망과 ViT 계층의 차이

합성곱 신경망은 일부만 학습하고 이를 추론해 이미지 전체의 특징을 추출 그러나 vit는 셀프 어텐션으로 모든 이미지 패치가 학습에 관여함.

→ 어텐션 거리를 계산하여 오직 한 개의 vit 레이어로 전체 이미지 정보를 쉽게 표현할 수 있음. 더 작은 모델로도 높은 성능을 얻을 수 있음.

단, 이미지 처리를 위해 이미지 크기를 위한 전처리가 필요함. 이미지 변환에는 취약함.

***귀납적 편향:** 해당 모델이 가지는 구조와 매개 변수들이 데이터에 적합한 가정을 하고 있음을 말함.

일반화 성능 향상을 위한 모델의 가정을 의미함. 이미지 데이터는 공간적 관계를 잘 표현하여 지역적 편향을 가진 합성곱 신경망 모델이 많이 사용됨. 반대로 시계열은 순환 신경망으로 순차적 편향을 띰.

그러나 vit는 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 일반화된 관계를 학습하기 때문에 귀납적 편향이 거의 없음. 대용량 데이터에 대해 이미지 특징들의 관계를 잘 학습함.

***구조**

1. 패치 임베딩: 각 패치를 벡터 형태로 변환함

작은 패치로 분할하기 위해 이미지 크기를 맞추는 전처리가 수행되어야 함. 보통 정방향으로 조절.

전체 이미지를 패치 크기로 분할해 시퀀셜 배열을 만듦. 이를 위해 커널 크기와 간격을 설정해야 함. 하이퍼 파라미터를 활용해 지정함. 왼→오. 위→아래로 배열하고 이 배열의 가장 왼쪽에 분류 토큰을 추가해서 전체 이미지를 대표하는 벡터로 예측하는 데 사용하도록 함.

위치 임베딩을 사용해 인접한 패치 간의 관계 학습함. 이후 계층 정규화를 적용하면 패치 임베딩이 만들어짐.

2. 인코더 : 각 패치와의 관계를 학습함

각 패치를 벡터로 변환 한 것이 입력으로 들어와 쿼리, 키 값, 임베딩의 관계를 학습함. 분류 토큰이라 불리는 특별한 패치의 특징 벡터를 추출함. 이 분류 토큰만을 활용해 통해 다양한 이미지 분류 및 검색 문제 해결.

1. 이미지의 특징을 추출한 후 분류, 회귀로 작업에 맞는 출력값으로 변환해 활용함.

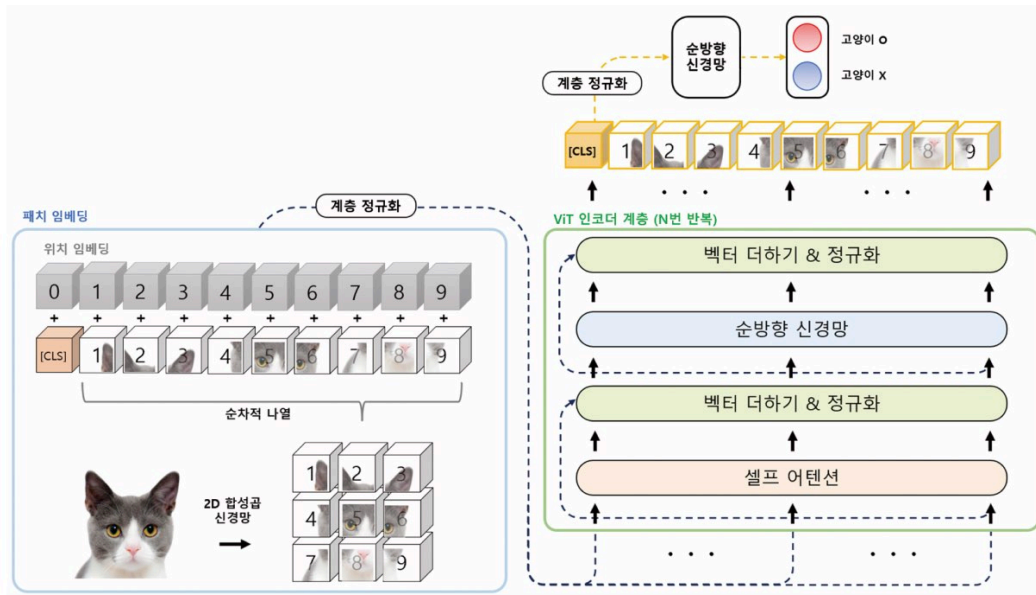


그림 10.4 ViT 모델 구조

스윈 트랜스포커

계층적 특징 맵을 구성해 1차 계산 복잡도를 낮도록함

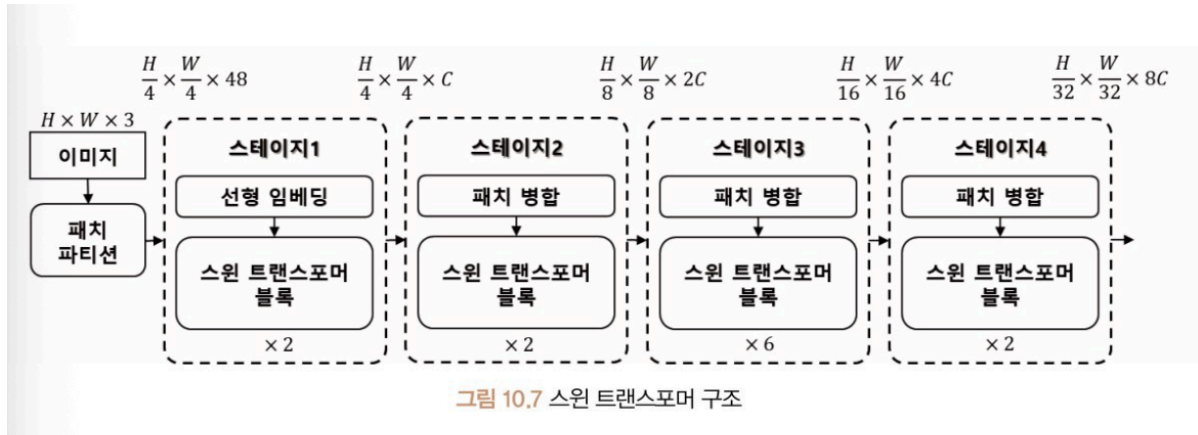
→ 시프트 윈도우 사용하여 일정한 크기의 패치로 분할 후 이를 겹쳐 쌓아 더 큰 효율성을 제공함.

다양한 크기와 종횡비에 적용 가능하다는 장점이 있음

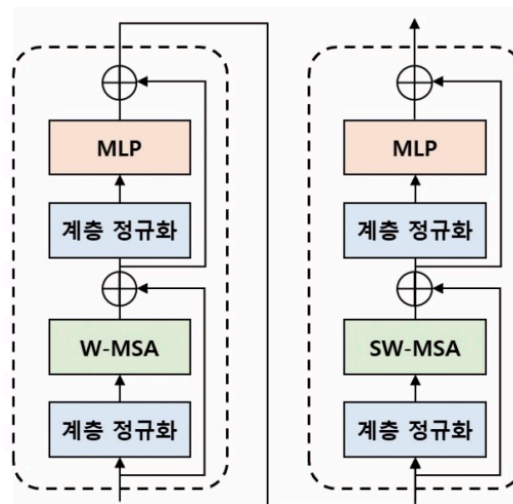
→ 로컬 윈도우를 활용해 물체의 크기나 해상도를 계층적으로 학습하여서 작은 고정된 크기의 윈도우임.

*구조

패치 파티션 → 선형 임베딩, 스윈 트랜스포머 블록, 패치병합의 반복



1. 패치 파티션: 입력 이미지를 작은 사각형 패치로 분할, 이후 패치 병합을 통해 인접한 패치들끼리는 정보를 저차원으로 축소함.
2. 스윈 트랜스포머:



w-msa란 로컬 윈도우를 사용하는 멀티 헤드 어텐션 이미지에서 일정한 크기의 윈도우 영역 설정. 해당 영역 내 픽셀 간의 어텐션을 계산함. 1차 복잡도로 줄여 연산량을 줄여줌.

s-msa 윈도우 사이의 연결성 구축. 관계 정보 추출. 순환 시프트(이동한 위치의 정보를 이전 위치에서 갖고 옴) 와 텐션 마스크(로컬 윈도우가 이동한 위치에서 연산을 수행하지 않도록 막음) 를 활용함. 이후 역순환으로 로컬 윈도우 구조로 복원함

상대적 위치 편향을 고려한다는 것 로컬 윈도우 안에 있는 패치 간의 상대적인 거리를 임베딩함

cls 토큰 사용하지 않음. 평균값으로 분류 진행

Cvt

▶ 핵심 아이디어

- CNN의 계층적 구조(Hierarchical Architecture)를 ViT에 접목
- 합성곱 연산으로 이미지 지역 특징(Local Feature) 추출
- 셀프 어텐션으로 전역 특징(Global Feature) 결합
- 결과: 적은 매개변수로 스윈 트랜스포머와 동등한 성능

표 10.4 ViT와 Cvt 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
Cvt	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

▶ 합성곱 토큰 임베딩 (Convolutional Token Embedding)

- 2D 이미지 구조를 보존하면서 패치를 생성
- 겹치는(overlapping) 합성곱으로 인접 패치 간 정보 공유
- 위치 임베딩 없이도 다양한 해상도 처리 가능

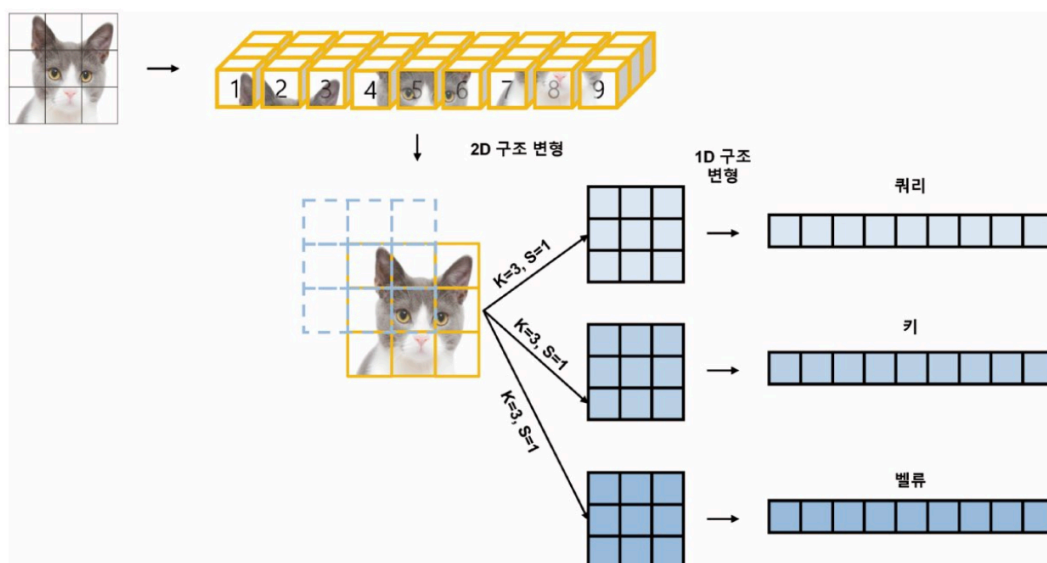


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

▶ 어텐션에 대한 합성곱 임베딩 (Conv Projection for Attention)

- 쿼리·키·값을 선형이 아닌 합성곱으로 임베딩
- 키·값의 stride를 크게 설정해 차원을 축소 (연산량 감소)
- 예) 쿼리: 9개 패치, 키·값: 4개 패치 → 어텐션 맵 9×4
→ 출력은 입력과 동일한 시퀀스 길이 유지

▶ CvT 모델 구조 (3개 스테이지)

스테이지0: CvtEmbeddings(Conv 7×7 , stride 4) + CvtLayer $\times$ 1

스테이지1: CvtEmbeddings(Conv 3×3 , stride 2) + CvtLayer $\times$ 1

스테이지2: CvtEmbeddings(Conv 3×3 , stride 2) + CvtLayer $\times$ 16

- 스테이지 진행 시 이미지 크기 4배·8배·16배 감소, 채널 차원 증가
- 풀링 없음: 합성곱이 공간 정보 압축 역할 수행

▶ 실습 모델: microsoft/cvt-21

- ImageNet-1k로 사전 학습, 입력 224×224
- 전처리 시 shortest_edge 기준으로 리사이즈

▶ 실습 결과

Epoch Training Loss Val Loss F1-Score

1	0.7737	0.3269	0.8914
2	0.7008	0.2596	0.9172
3	0.6438	0.2460	0.9173

- 최종 F1: 0.9173 (Swin 0.9159 대비 소폭 개선)
- 일반화 성능 최고: CvT
- 모델 크기 대비 성능 최고: Swin Transformer
- F1-Score 최고: ViT
- 선택 기준: 풀어야 하는 문제와 사용처에 따라 적절히 선택